

UT.5 TRATAMIENTO DE DATOS

1. INTRODUCCIÓN	2
2. INSERCIÓN DE REGISTROS EN LA BASE DE DATOS: INSERT	2
2.1 COPIA DE REGISTROS DE OTRA TABLA	6
2.2 MANEJO DE ERRORES COMUNES EN EL INSERT	8
3. MODIFICACIÓN DE REGISTROS EN UNA TABLA: UPDATE	9
4. ELIMINACIÓN DE REGISTROS DE UNA TABLA: DELETE	12
5. TRANSACCIONES EN LA BASE DE DATOS	14
6. BLOQUEOS EN ORACLE. ACCESO SIMULTÁNEO A LOS DATOS.	18
6.1 BLOQUEOS COMPARTIDOS Y EXCLUSIVOS	18
6.2 INTERBLOQUEOS (DEADLOCKS)	19

1.INTRODUCCIÓN

Las bases de datos no tienen razón de ser sin la posibilidad de hacer operaciones para el tratamiento de la información almacenada en ellas. Por operaciones de tratamiento de datos se deben entender las acciones que permiten añadir información en ellas, modificarla o bien suprimirla.

Pero la información no se puede almacenar en la base de datos sin tener en cuenta que debe seguir una serie de requisitos en las relaciones existentes entre las tablas que la componen. Todas las operaciones que se realicen respecto al tratamiento de los datos deben asegurar que las relaciones existentes entre ellos se cumplan correctamente en todo momento.

Por otro lado, la ejecución de las aplicaciones puede fallar en un momento dado y eso no debe impedir que la información almacenada sea incorrecta. O incluso el mismo usuario de las aplicaciones debe tener la posibilidad de cancelar una determinada operación y dicha cancelación no debe suponer un problema para que los datos almacenados se encuentren en un estado fiable.

Todo esto requiere disponer de una serie de herramientas que aseguren esa fiabilidad de la información, y que además puede ser consultada y manipulada en sistemas multiusuario sin que las acciones realizadas por un determinado usuario afecte negativamente a las operaciones de los demás usuarios.

Para todo ello, el lenguaje de manipulación de datos o DML (Data Manipulation Language) es la parte central de SQL. Actúa de dos formas en la base de datos:

1. Cuando agrega, modifica o elimina registros de las tablas de la base de datos
2. Como transacción, la cual consiste en una colección de sentencias DML que forman una unidad lógica de trabajo.

2.INSERCIÓN DE REGISTROS EN LA BASE DE DATOS: INSERT



La sentencia **INSERT** permite la inserción de nuevas filas o registros en una tabla existente. Su sintaxis es:

```
INSERT INTO nb_tabla [ ( columna1[ , columna2 ...] ) ] VALUES  
(valor1 [ , valor2...]);
```

Donde nb_tabla será el nombre de la tabla en la que quieras añadir nuevos registros. En la lista de campos (columna1, ...) se indicarán los campos de dicha tabla en los que se desea escribir los nuevos valores. Es posible omitir la lista de campos si se indican todos los valores de cada campo (valor1, ...) en el orden en el que se encuentran en la tabla. Los valores de datos de tipo carácter y fecha se encerrarán entre comillas simples.

Ejemplo: Inserta en la tabla DEPARTAMENTO un registro con la siguiente información:

- Código de departamento: VENZN
- No tiene director.
- Depende del departamento DIRGE
- Pertenece al centro de trabajo DIGE
- Nombre: Ventas zona norte
- Tiene un presupuesto anual de 8250000€
- Tipo de dirección: 'F'

```
INSERT INTO DEPARTAMENTO (CODDEP, CODEMPDIR, CODDEPDEP, CODCEN,  
NOMDEP, PREANU, TIDIR)  
VALUES ('VENZN', NULL, 'DIRGE', 'DIGE', 'Ventas zona norte',  
8250000, 'F');
```

Para insertar valores nulos tenemos dos posibilidades:

- **Método implícito:** Consiste en omitir la columna de la lista. Ejemplo:

```
INSERT INTO CENTRO (CODCEN, NOMCEN, DIRCEN, POBCEN)  
VALUES ('OFZN', 'Oficinas Zona Norte', 'Paseo de Gracia 120',  
'Barcelona');
```

Al haber omitido la columna CODEMPDIR, se crea el registro con su valor a NULL.

- **Método explícito:** Se especifica el valor de la columna con la palabra null o con el string vacío '' en la lista de valores (En **Oracle** un string vacío (' ') se trata como NULL. Sin embargo, en **MySQL, PostgreSQL o SQL Server**, un string vacío y un valor NULL son cosas totalmente distintas. Ahí, el string vacío es un valor de longitud cero, mientras que NULL es la ausencia de valor.).

Ejemplo:

```
INSERT INTO CENTRO (CODCEN, CODEMPDIR, NOMCEN, DIRCEN, POBCEN)  
VALUES ('OFZO', NULL, 'Oficinas Zona Oeste', 'C/ Mayor 55',  
'Cáceres');
```

```
INSERT INTO CENTRO (CODCEN, CODEMPDIR, NOMCEN, DIRCEN, POBCEN)
```

```
VALUES ('MARKG', '', 'Oficinas Zona Centro', 'Paseo de la Castellana 214', 'Madrid');
```

Si la lista de campos indicados no se corresponde con la lista de valores, se obtendrá un error en la ejecución.

INSERCIÓN DE FECHAS

- **SYSDATE:** Cuando se inserta un valor de fecha normalmente se inserta con el formato por defecto. Si queremos convertirlo a otro formato en la inserción se utilizará la función TO_DATE .

Ejemplo: Insertar en la tabla EMPLEADO llamado 'Gómez Vendedor, Mariano', del departamento 'VENZC', sin teléfono asignado, con fecha de ingreso en la empresa la fecha de hoy fecha de nacimiento 5 de marzo de 1991. Su NIF es 98765432F, no tiene hijos, su salario mensual es de 2900€ y cobra 250€ en comisiones.

```
INSERT INTO EMPLEADO
(CODEMP, CODDEP, FECINEMP, FECNAEMP, NIFEMP, NOMEMP, NUMHI, SALEMP,
COMEMP)
VALUES (31, 'MARKG', SYSDATE, TO_DATE('05/03/1991', 'DD/MM/YYYY'),
'98765432F', 'Gómez Vendedor, Mariano', 0, 2900, 250);
```

Y recuerda que si intentas insertar un valor en una columna que es FK (clave ajena) y ese valor no existe en la tabla maestra, la base de datos rechazará la inserción para mantener la integridad.

INSERCIÓN DE VALORES POR VARIABLES DE SUSTITUCIÓN

Es posible crear sentencias INSERT dinámicas en SQL*Plus utilizando variables de sustitución.

Ejemplo:

Insertar registros en la tabla HABILIDAD usando variables de sustitución.

```
INSERT INTO HABILIDAD
VALUES (&CODHAB, &DESHAB);
```

Introduzca un valor para CODHAB: 'LOGIS'

Introduzca un valor para DESHAB: 'Logística'

Si el valor es de tipo **carácter** (string), la variable debe ir entre comillas simples en la sentencia si el usuario no las va a teclear:

```
VALUES ( '&CODHAB' , '&DESHAB' );
```

Si el usuario escribe LOGIS (sin comillas) en el prompt y la sentencia no tiene las comillas, SQL dará error.

VALORES POR DEFECTO

Si una columna se definió con un valor por defecto al crear la tabla (ej: FECHA_ALTA DATE DEFAULT SYSDATE), puedes usar la palabra clave DEFAULT en el INSERT:

```
INSERT INTO EMPLEADO (CODEMP, NOMEMP, FECINEMP)
VALUES (45, 'Ana López', DEFAULT);
-- Esto insertará automáticamente la fecha de hoy sin necesidad de llamar a SYSDATE.
```

Por otro lado, si una columna tiene un valor DEFAULT definido y te la saltas en el INSERT: **se insertará el valor por defecto**, no un NULL (método implícito). ¿Cuándo incluirla?:

1. Legibilidad (Documentar tu intención)

Cuando otro programador (o tú mismo en el futuro) lee el código, ver la palabra DEFAULT deja claro que **sabes** que esa columna existe y que **quieres** que tome su valor predefinido. Si simplemente falta la columna, alguien podría pensar que se te ha olvidado incluirla por error.

2. Para no tener que listar todas las columnas

Si quieres insertar un registro en una tabla con muchas columnas y quieres usar el valor por defecto en algunas de ellas, pero **no quieres escribir la lista de columnas** al principio (porque vas a dar valores para todas), necesitas la palabra clave DEFAULT.

Recuerda: Si omites la lista de campos, estás obligado a pasar un valor para cada columna de la tabla en orden.

3. En sentencias dinámicas

A veces, en aplicaciones que generan SQL automáticamente, es más fácil mantener una estructura fija donde siempre se mencionan todas las columnas y simplemente se decide si enviar un valor concreto, un NULL o un DEFAULT.

LA INSERCIÓN MÚLTIPLE

A veces no queremos ejecutar varias sentencias INSERT distintas, sino una sola que haga todo el trabajo, lo cual es más eficiente para el rendimiento del servidor. En muchos gestores (aunque en Oracle no se ha implementado hasta la versión 23c), puedes separar los registros por comas:

```
INSERT INTO HABILIDAD (CODHAB, DESHAB)
VALUES
    ('JAVA', 'Programación Java'),
    ('SQL', 'Consultas de Base de Datos'),
    ('PYTHON', 'Análisis de datos');
```

2.1 COPIA DE REGISTROS DE OTRA TABLA

Se puede usar la sentencia INSERT para agregar filas a una tabla donde los valores se derivan de otras tablas existentes, usando en vez de la cláusula VALUES una subconsulta, para ello se debe:

1. Escribir el comando INSERT con una subconsulta.
2. No usar la cláusula VALUES.
3. Observar que coincida el número de columnas de INSERT con el de la subconsulta

Sintaxis:

```
INSERT INTO nb_tabla [ columna ( , columna... ) ] SUBQUERY;
```

Ejemplo: Supongamos que tenemos una tabla MANAGER con una estructura idéntica a la tabla conocida EMPLEADO, y queremos rellenar la tabla MANAGER con el contenido de la tabla EMPLEADO en la que el trabajador sea MANAGER (responsable de un departamento).

```
INSERT INTO MANAGER
(SELECT E.*
FROM EMPLEADO E
JOIN DEPARTAMENTO D ON E.CODEMP = D.CODEMPDIR);
```

```
INSERT INTO MANAGER
(SELECT *
FROM EMPLEADO
WHERE CODEMP IN
(SELECT DISTINCT CODEMPDIR
FROM DEPARTAMENTO));
```

```

INSERT INTO MANAGER
(
    SELECT E.*
    FROM EMPLEADO E
    WHERE EXISTS
    (
        SELECT 1
        FROM DEPARTAMENTO D
        WHERE D.CODEMPDIR = E.CODEMP
    )
);

```

Cualquiera de las opciones es válida. Por supuesto es posible insertar registros en una tabla a través de una subconsulta que lee datos de varias tablas. Es una de las operaciones más potentes y comunes en el mundo real, especialmente en procesos de **carga de datos**, **generación de informes** o cuando necesitas crear "fotos" de la información en un momento determinado. Permite:

- **Desnormalización controlada:** A veces, para mejorar el rendimiento de una aplicación o un informe, necesitas una tabla que tenga datos que normalmente están separados (ej. el nombre del empleado junto con el nombre de su departamento).
- **Consolidación de datos:** Permite resumir información. Por ejemplo, insertar en una tabla de "Histórico de ventas" el total vendido por cada cliente, sumando datos de facturas, líneas de factura y productos.
- **Migración y Transformación:** Es la base de los procesos ETL (*Extract, Transform, Load*). Extraes de varios sitios, transformas el formato y lo cargas en el destino.

Imagina que queremos rellenar una tabla llamada LISTA_CONTACTOS que debe contener el nombre del empleado, su departamento y el nombre de su jefe (para saber a quién llamar si pasa algo).

```

INSERT INTO LISTA_CONTACTOS (NOM_EMPLEADO, NOMBRE_DEP, NOM_JEFE)
SELECT
    E.NOMEMP,
    D.NOMDEP,
    J.NOMEMP AS NOM_JEFE
FROM EMPLEADO E
JOIN DEPARTAMENTO D ON E.CODDEP = D.CODDEP
LEFT JOIN EMPLEADO J ON D.CODEMPDIR = J.CODEMP; -- Buscamos al jefe
del departamento

```

2.2 MANEJO DE ERRORES COMUNES EN EL INSERT

Incluso con la sintaxis correcta, el motor de la base de datos puede rechazar la inserción. Estos son los errores más habituales:

Violación de clave primaria (Unique constraint)

- **Error:** Intentas insertar un valor que ya existe en una columna marcada como PRIMARY KEY o UNIQUE.
- **Mensaje típico:** *ORA-00001: unique constraint (...) violated.*
- **Solución:** Verifica si el registro ya existe o si estás usando un contador/secuencia que se ha quedado atrás.

Violación de integridad referencial (Foreign key)

- **Error:** Intentas insertar un valor en una clave ajena (FK) que no existe en la tabla maestra.
- **Ejemplo:** Insertar un empleado en el departamento 'MARKETING' si en la tabla DEPARTAMENTO ese código no existe.
- **Mensaje típico:** *parent key not found.*

Error de tipo de dato o longitud

- **Error:** Intentas meter un texto de 50 caracteres en un campo VARCHAR2(20), o un texto en un campo numérico.
- **Mensaje típico:** *value too large for column o invalid number.*

Violación de restricción check

- **Error:** La tabla tiene una regla (ej: SALARIO > 0) e intentas insertar un valor que no la cumple.
- **Mensaje típico:** *check constraint (...) violated.*

3.MODIFICACIÓN DE REGISTROS EN UNA TABLA: UPDATE



Para modificar el contenido de los registros de una tabla se hace mediante la sentencia UPDATE, la cual permite además modificar varios registros a la vez. Sintaxis:

```
UPDATE nb_tabla  
SET columna1 = valor1 [, columna2 = valor2 ...]  
[WHERE condición];
```

Como puede verse, la cláusula WHERE es opcional, y permite filtrar los registros que se verán afectados por la actualización, que serán los que cumplan la condición. Si no especificamos el WHERE, la modificación se hará en todos los registros de la tabla.

¡CUIDADO!: Un error muy común es olvidar la cláusula WHERE. Si ejecutas UPDATE EMPLEADO SET SALEMP = 0;, ¡toda la empresa dejará de cobrar! Se recomienda hacer siempre un SELECT previo con el mismo WHERE para verificar qué filas vas a modificar.

Ejemplo 1: Transferir al empleado 4 al departamento VENZL:

```
UPDATE EMPLEADO  
SET CODDEP = 'VENZL'  
WHERE CODEMP = 4;
```

Ejemplo 2: Subir el salario un 5% y la comisión un 10% a los empleados del departamento MARKG:

```
UPDATE EMPLEADO  
SET SALEMP = SALEMP * 1.05,  
COMEMP = COMEMP * 1.10  
WHERE CODDEP = 'MARKG';
```

Si queremos aplicar la subida salarial a toda la empresa simplemente quitamos la cláusula WHERE de la sentencia anterior.

Cuando termina la ejecución de una sentencia UPDATE, se muestra la cantidad de registros (filas) que han sido actualizadas, o el error correspondiente si se ha producido algún problema.

También podemos usar UPDATE para "borrar" información de una columna específica (siempre que la columna permita nulos).

- **Ejemplo:** Quitarle la comisión a un empleado porque ha cambiado de puesto:
`UPDATE EMPLEADO SET COMEMP = NULL WHERE CODEMP = 10;`

Si al realizar la modificación se intenta actualizar un registro con un determinado valor que viola una restricción de identidad se produce un error. El siguiente ejemplo fallará porque no existe un departamento con código ADMZN:

```
UPDATE EMPLEADO
SET CODDEP = 'ADMZN'
WHERE CODEMP = 5;
```

Error report -

ORA-02291: integrity constraint (UT4EMPRESA.FK_EMP_DEP) violated -
parent key not found

Además de la violación de clave ajena podemos encontrar los siguientes errores:

- **Violación de Check Constraint:**
 - *Ejemplo:* Intentar poner un salario negativo si existe una restricción CHECK (SALEMP > 0).
- **Violación de Unicidad (Unique Constraint):**
 - *Ejemplo:* Intentar cambiar el NIF de un empleado por uno que ya existe en otro registro.
- **Actualización de Clave Primaria:**
 - Aunque técnicamente es posible, **nunca** se debe actualizar una PRIMARY KEY si esta tiene tablas relacionadas (claves ajenas), ya que rompería la integridad o requeriría actualizaciones en cascada.

Las **subconsultas multicolumna** pueden ser implementadas en la cláusula SET de una sentencia UPDATE. Sintaxis:

```
UPDATE nb_tabla
SET (columna,columna...) =
(SELECT columna, columna, ... FROM nb_tabla WHERE condición)
[WHERE condición];
```

Ejemplo: Asignar al empleado 4 el mismo departamento y salario que tiene el empleado 3.

```
UPDATE EMPLEADO
SET (CODDEP, SALEMP) =
(SELECT CODDEP, SALEMP
```

```
FROM EMPLEADO
WHERE CODEMP = 3)
WHERE CODEMP = 4;
```

Hay un comportamiento de Oracle que hay que conocer: **Si la subconsulta del SET no devuelve ninguna fila, Oracle intentará actualizar las columnas con NULL.**

Si el empleado 3 no existiera en la base de datos, el empleado 4 vería su departamento y salario borrados (NULL). Si esas columnas fueran NOT NULL, la sentencia fallaría con un error. Asegúrate siempre de que la subconsulta del SET devuelva exactamente una fila. Si la subconsulta no devuelve nada, Oracle intentará asignar valores NULL a esas columnas. Si quieres evitar esto, puedes añadir el mismo filtro en el WHERE principal de la sentencia.

```
UPDATE EMPLEADO
SET (CODDEP, SALEMP) = (SELECT CODDEP, SALEMP FROM EMPLEADO WHERE
CODEMP = 3)
WHERE CODEMP = 4
AND EXISTS (SELECT 1 FROM EMPLEADO WHERE CODEMP = 3);
```

ACTUALIZACIONES CON SUBCONSULTAS CORRELACIONADAS: Podemos utilizar subconsultas correlacionadas en la cláusula WHERE para filtrar los registros que se actualizarán. Por ejemplo: Actualizar el presupuesto de los departamentos subiéndolo un 10% solo para aquellos centros que están en 'Murcia'.

```
UPDATE DEPARTAMENTO D
SET PREANU = PREANU * 1.10
WHERE EXISTS (
    SELECT 1
    FROM CENTRO C
    WHERE C.CODCEN = D.CODCEN
    AND C.POBCEN = 'Murcia'
);
```

4. ELIMINACIÓN DE REGISTROS DE UNA TABLA: DELETE



La sentencia DELETE es la que permite eliminar o borrar registros de una tabla. Esta es la sintaxis que debes tener en cuenta para utilizarla:

```
DELETE [FROM ] nb_tabla [WHERE condición];
```

Como en el caso del UPDATE, el WHERE determina los registros que son eliminados. Ojo, **si se omite la cláusula WHERE se eliminan todos los registros de la tabla**, aunque la tabla seguirá existiendo con la estructura que tenía hasta el momento.

https://youtu.be/i_cVJglz_Cs?si=pmHeDkeSP1H8GJDM

Ejemplo 1:

Borrar de la tabla HIJO todos los hijos de empleados cuyo primer apellido sea 'Moreno':

```
DELETE
FROM HIJO
WHERE UPPER(NOMHI) LIKE 'MORENO%';
```

Se pueden eliminar registros basados en otra tabla utilizando subconsultas en la sentencia DELETE.

Ejemplo 2:

Eliminar de la tabla HIJO todos los registros correspondientes al empleado con NIF 55001122L.

```
DELETE
FROM HIJO
WHERE CODEMP =
(SELECT CODEMP
FROM EMPLEADO
WHERE NIFEMP = '55001122L');
```

Si se intenta borrar un registro con un valor relacionado con otra tabla dará un error de integridad, ya que, no se pueden eliminar registros que contengan una clave primaria, usada como clave foránea en otra tabla.

Ejemplo 3:

Intenta borrar de la tabla EMPLEADO el registro correspondiente al código 14 y verás como la base de datos no lo permite.

```
DELETE  
FROM EMPLEADO  
WHERE CODEMP = 14;
```

Dos tablas pueden ser relacionadas entre ellas si tienen en común uno o más campos, que reciben el nombre de clave ajena. La restricción de integridad referencial requiere que haya coincidencia en todos los valores que deben tener en común ambas tablas. Cada valor del campo que forma parte de la integridad referencial definida, debe corresponderse, en la otra tabla, con otro registro que contenga el mismo valor en el campo referenciado. El gestor de la base de datos asegura la consistencia de la información impidiendo que queden claves ajenas “huérfanas” al realizar cualquier tipo de modificación de la información.

Para manejar esto tenemos dos opciones:

1. **Borrado manual:** Borrar primero los registros "hijos" (en la tabla HIJO) y luego el "padre" (en EMPLEADO).
2. **Configuración de la tabla:** Mencionar que la restricción FK puede definirse como ON DELETE CASCADE.

Nota técnica: "Si la clave ajena se definió con ON DELETE CASCADE, al borrar al empleado 14, Oracle borraría automáticamente a todos sus hijos sin dar error. Si se definió con ON DELETE SET NULL, pondría a los hijos como huérfanos con valor NULL".

UTILIZACIÓN DE SUBCONSULTAS CORRELACIONADAS EN UN DELETE: Siguiendo la lógica del UPDATE, es muy común querer borrar registros de una tabla basándonos en una condición compleja de otra tabla.

- Ejemplo: Borrar los hijos de aquellos empleados que trabajan en el departamento 'MARKG'.

```
DELETE FROM HIJO H
WHERE EXISTS (
    SELECT 1
    FROM EMPLEADO E
    WHERE E.CODEMP = H.CODEMP
    AND E.CODDEP = 'MARKG'
);
```

5. TRANSACCIONES EN LA BASE DE DATOS

El servidor ORACLE asegura consistencia en los datos basándose en las transacciones. Se entiende por **TRANSACCIÓN** al periodo de tiempo desde que el usuario ejecuta la **primera sentencia DML** (INSERT, UPDATE, DELETE, MERGE) o con un SELECT . . . FOR UPDATE hasta que ocurre uno de los siguientes eventos:

- Un comando **COMMIT** o **ROLLBACK**.
- Un comando DDL o DCL (Antes de ejecutar un DDL, Oracle lanza un COMMIT silencioso **independientemente de si el comando DDL termina con éxito o da error**. Si el DDL funciona, lanza otro COMMIT después).
- Al producirse ciertos errores.
- Finaliza la sesión de forma normal: el comportamiento depende de la herramienta cliente. En **SQL*Plus**, ejecutar EXIT lanza un COMMIT automático. En **SQL Developer**, cerrar la conexión o la sesión realiza un **ROLLBACK** de los cambios no confirmados, por lo que nunca debes asumir que cerrar la ventana guarda tu trabajo.
- Un fallo o caída del sistema. Al reiniciar, la base de datos hace un ROLLBACK automático de todas las transacciones que no habían sido confirmadas.

Una transacción es una unidad lógica y atómica (no se puede dividir) de trabajo que contiene una o más sentencias SQL. Las transacciones agrupan sentencias SQL de tal manera que a todas ellas se le aplica una operación COMMIT, que podríamos traducir como aplicadas o guardadas en la base de datos, o bien a todas ellas se les aplica la acción ROLLBACK, que interpretamos como deshacer las operaciones que deberían hacer sobre la base de datos. Por tanto, mientras que sobre una transacción no se haga COMMIT, los resultados de ésta pueden deshacerse.

Las propiedades ACID

Para garantizar la fiabilidad del sistema, todas las transacciones deben cumplir las propiedades ACID:

- **Atomicidad**: "Todo o nada". Las sentencias de la transacción se aplican todas juntas o no se aplica ninguna.

- **Consistencia:** La base de datos pasa de un estado válido a otro estado válido, respetando todas las reglas e integridades (claves ajenas, restricciones, etc.).
- **Isolation (Aislamiento):** Las transacciones concurrentes no interfieren entre sí. Un usuario no ve los datos "a medias" de otro.
- **Durability (Durabilidad):** Una vez hecho el COMMIT, los cambios son permanentes y sobreviven a fallos físicos o del servidor.

Consistencia de lectura y segmentos UNDO

Mientras un usuario realiza cambios (sin hacer COMMIT), los demás usuarios que consulten la tabla no verán los datos modificados, sino los datos originales.

Esto se logra gracias a los **segmentos de UNDO** (deshacer), donde Oracle guarda temporalmente la "fotografía" anterior de los datos. Esto garantiza que nadie lea información inconsistente mientras otra persona la está modificando.

Puntos de restauración (SAVEPOINT)

Un punto de restauración (SAVEPOINT) es un marcador intermedio declarado por el usuario en el contexto de una transacción. Los puntos de restauración dividen una transacción grande en pequeñas partes.

Si usas puntos de restauración en una transacción larga, tendrás la opción de deshacer los cambios efectuados por la transacción antes de la sentencia actual en la que se encuentre, pero después del punto de restauración establecido. Así, si se produce un error, no es necesario rehacer todas las sentencias de la transacción completa, sino sólo aquellos posteriores al punto de restauración.

Sentencia	Descripción
COMMIT;	- Finaliza la transacción actual haciendo que todos los cambios pendientes pasen a ser permanentes.
SAVEPOINT nombre;	- Realiza una marca dentro de la transacción en curso.
ROLLBACK;	- Finaliza la transacción descartando todos los cambios pendientes desde su inicio.
ROLLBACK TO SAVEPOINT nombre;	- deshace los cambios realizados desde el savepoint indicado, pero la transacción continúa activa.

Ejemplo:

```
-- Consultamos el estado inicial (Esto NO inicia una transacción)
SELECT * FROM EMPLEADO ORDER BY CODEMP;

-- 1. Inicio de la transacción con el primer DML
INSERT INTO EMPLEADO VALUES
(31, 'ADMZS', NULL, SYSDATE, TO_DATE ('2000-01-01', 'YYYY-MM-DD'),
'1212121A', 'Martínez Mejor, Ana', 0, 3750, NULL);

-- 2. Creamos el primer punto de guardado
SAVEPOINT A;

-- 3. Hacemos dos modificaciones más
UPDATE EMPLEADO SET EXTELEMP = '91919191' WHERE CODEMP = 30;

INSERT INTO EMPLEADO VALUES
(32, 'ADMZS', NULL, SYSDATE, TO_DATE ('1995-10-01', 'YYYY-MM-DD'),
'7474747C', 'González Gestor, Manuel', 0, 3100, 200);

-- 4. Creamos el segundo punto de guardado
SAVEPOINT B;

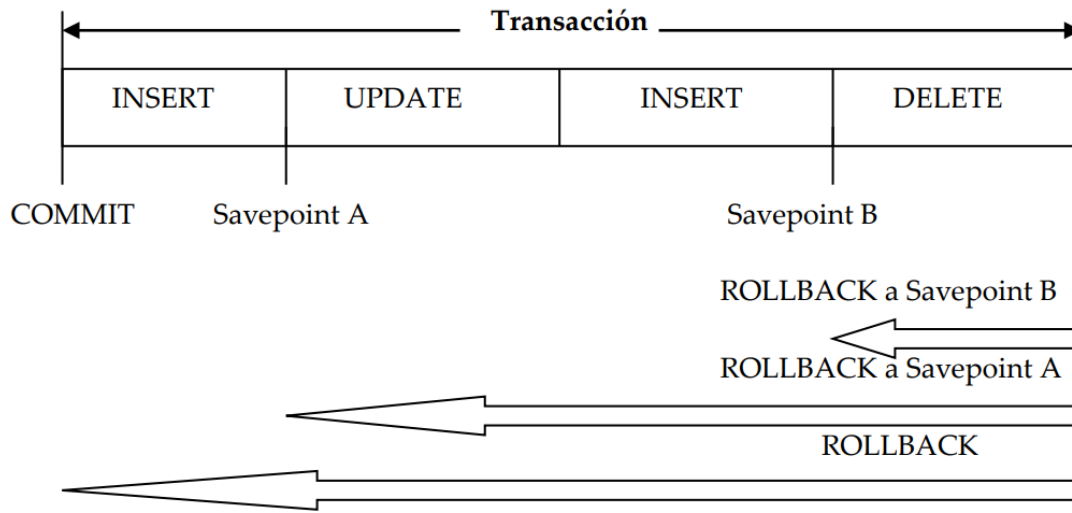
-- 5. Borrarnos el primer registro insertado
DELETE FROM EMPLEADO
WHERE CODEMP = 31;

-- 6. Deshace la última acción (el DELETE). El empleado 31 vuelve a
existir.
ROLLBACK TO SAVEPOINT B;

-- 7. Deshace los cambios hasta la marca A (el segundo INSERT y el
UPDATE).
ROLLBACK TO SAVEPOINT A;

-- 8. Deshace el primer INSERT y finaliza la transacción.
-- La tabla queda exactamente como estaba en el estado inicial.
ROLLBACK;
```

Comprueba el estado de la tabla después de cada instrucción.



Ventajas de COMMIT y ROLLBACK.

1. Aseguran la consistencia de los datos.
2. Pueden visualizar los cambios sobre los datos antes de hacerlos permanentes.
3. Agrupan lógicamente las tareas relacionadas entre sí.

Estado de los datos antes de COMMIT y ROLLBACK.

1. El estado previo de los datos puede ser recuperado porque el afectado por los cambios es el búfer de la base de datos.
2. El usuario actual debe revisar los resultados de sus operaciones DML usando la sentencia SELECT.
3. Otros usuarios no pueden ver los resultados de las sentencias DML ejecutadas por el usuario actual.
4. Las filas afectadas son bloqueadas, otros usuarios no podrán cambiar los datos pertenecientes a esas filas.

Estado de los datos después del COMMIT:

1. Los datos manipulados son escritos en la BD.
2. Los datos anteriores se pierden definitivamente.
3. Todos los usuarios podrán ver los resultados.
4. Se liberan los bloqueos aplicados a las filas afectadas; esas filas están ahora disponibles para que otros usuarios las usen.
5. Se borran todos los SAVEPOINT.

Estado de los datos después de ROLLBACK.

1. Se descartan todos los cambios pendientes.
2. Se restaura el estado previo de los datos.
3. Los cambios de los datos se deshacen.
4. Se levantan todos los bloqueos sobre las filas afectadas

6. BLOQUEOS EN ORACLE. ACCESO SIMULTÁNEO A LOS DATOS.

En una base de datos a la que accede un solo usuario, un dato puede ser modificado sin tener en cuenta que otros usuarios puedan modificar el mismo dato al mismo tiempo. Sin embargo, en una base de datos multiusuario, las sentencias contenidas en varias transacciones simultáneas pueden actualizar los datos simultáneamente. Las **transacciones ejecutadas simultáneamente**, deben generar **resultados consistentes**.

Los **bloqueos** son mecanismos que **previenen conflictos entre las transacciones** que acceden a los mismos recursos, bien sea un objeto de usuario (tabla o registro), o un objeto del sistema no visible por los usuarios (como estructuras de datos compartidas y registros del diccionario de datos), garantizando la **integridad y consistencia** de la base de datos (propiedades ACID).

El bloqueo de ORACLE es completamente automático y no requiere acción por parte del usuario. Previenen la interacción destructiva entre transacciones concurrentes, y es además un mecanismo automático que utiliza el nivel más bajo aplicable de restricción, por lo que ofrece además un grado más alto de concurrencia y máxima integridad de datos.

- Un registro es **bloqueado automáticamente cuando es modificado por un escritor**: Cuando una sentencia actualiza un registro, la transacción obtiene un bloqueo sólo para ese registro.
- **Un escritor de un registro bloquea a otro escritor concurrente del mismo registro**: Si una transacción está modificando una fila, un bloqueo del registro impide que otra transacción modifique el mismo registro simultáneamente.
- **Un lector nunca bloquea a un escritor**: Puesto que un lector de un registro no lo bloquea, un escritor puede modificar dicho registro. La única excepción es la sentencia `SELECT ... FOR UPDATE`, que es un tipo especial de sentencia `SELECT` que bloquea el registro que está siendo consultado.
- **Un escritor nunca bloquea a un lector**: Cuando un registro está siendo modificado, la base de datos proporciona al lector una vista del registro sin los cambios que se están realizando.

6.1 BLOQUEOS COMPARTIDOS Y EXCLUSIVOS

En general, la base de datos usa dos tipos de bloqueos: bloqueos exclusivos y bloqueos compartidos:

- **Exclusivo**: Previene la compartición de un recurso. La primera transacción que bloquea el recurso es la única que puede alterarlo, hasta liberar el bloqueo.
- **Compartido**: Varias transacciones pueden mantener este tipo de bloqueo sobre el mismo recurso al mismo tiempo, de ahí el nombre. Sin embargo, ninguna de ellas puede coexistir con un bloqueo exclusivo sobre ese mismo recurso. Es decir, el bloqueo compartido protege frente a operaciones que necesiten exclusividad total,

como los DDL, pero no impide que otras transacciones accedan al recurso de forma similar.

Un caso habitual de combinación de ambos tipos de bloqueo se da con SELECT ... FOR UPDATE: la transacción obtiene un **bloqueo exclusivo sobre la fila** consultada (para impedir que otra sesión la modifique) y simultáneamente un **bloqueo compartido sobre la tabla** (para impedir que otra sesión altere su estructura con un DDL, pero permitiendo que otras sesiones sigan haciendo DML sobre el resto de filas). El ejemplo práctico que se muestra a continuación ilustra el caso más frecuente: dos escritores intentando modificar el mismo registro:

- Abre dos sesiones (por ejemplo dos pestañas en SQL Developer) contra la misma conexión (mismo usuario).
 - **Sesión 1:**

```
UPDATE MULTAS SET IMPORTE = 500 WHERE REF = '5721369855';
```

```
-- OJO: No hagas COMMIT aún.
```

- **Sesión 2:**

```
UPDATE MULTAS SET IMPORTE = 1 WHERE REF = '5721369855';
```

```
-- Se queda en espera. La sesión está bloqueada.
```

- **Resolución:** En la Sesión 1 escribe COMMIT; y verás cómo la Sesión 2 se libera instantáneamente.

6.2 INTERBLOQUEOS (DEADLOCKS)

Un deadlock (interbloqueo) se produce cuando dos o más transacciones se bloquean mutuamente esperando recursos que la otra posee, creando un ciclo de espera que ninguna puede resolver por sí sola.

Ejemplo:

1. Usuario A bloquea Fila 1.
 2. Usuario B bloquea Fila 2.
 3. Usuario A intenta bloquear Fila 2 (espera a B).
 4. Usuario B intenta bloquear Fila 1 (espera a A).
- **Resultado:** Oracle detecta esto automáticamente y elige a una de las sesiones involucradas en el interbloqueo para ser la "víctima" (suele ser la sesión que cierra el círculo del bloqueo y lo hace insalvable). A esta sesión víctima se le lanza el error ORA-00060: interbloqueo detectado mientras se esperaba un recurso. Sin embargo, Oracle **solo deshace la última instrucción SQL** de esa sesión; es decir, la sentencia exacta que causó el nudo. Todo el trabajo que la sesión

víctima había realizado *antes* de lanzar la sentencia conflictiva **sigue intacto y activo**. Sus bloqueos anteriores siguen vigentes.

Ejemplo DEADLOCK:

- Abre dos sesiones (por ejemplo dos pestañas en SQL Developer) contra la misma conexión (mismo usuario).
 - **Sesión 1:**

```
UPDATE MULTAS
```

```
SET IMPORTE = 80
```

```
WHERE REF = '5721369855' ;
```

- **Sesión 2:**

```
UPDATE MULTAS
```

```
SET IMPORTE = 150
```

```
WHERE REF = '8652126792' ;
```

- **Sesión 1:**

```
UPDATE MULTAS
```

```
SET IMPORTE = 80
```

```
WHERE REF = '8652126792' ;
```

- **Sesión 2:**

```
UPDATE MULTAS
```

```
SET IMPORTE = 150
```

```
WHERE REF = '5721369855' ;
```

- En este momento una de las sesiones recibe un error ORA-00060 junto con un ROLLBACK automático de su última instrucción. El resto de instrucciones, junto con sus correspondientes bloqueos, siguen activos.